COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 10, 2026	**DATE SUBMITTED:**	July 14, 2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr, Jimlord Quejado

ACTIVITY NO. 1 **Algorithm, Flowchart and Pseudocode**

1. PROCEDURE OUTPUT

```
#include <iostream>
#include <cmath>
#include <string>

//Task 1
void numSwap();

//Task 2
double kelvinToFahrenheit(double kelvin);

//Task 3
void calculateDistance();
double distanceCalculator(double x1, double x2, double y1, double y2);

//TASK 4
class Triangle {
private:
    double sideA, sideB, sideC;
    double angleA, angleB, angleC;
    double totalAngle;

public:
    Triangle(double sA, double sB, double sC,
             double A, double B, double C);

    void setTriangle(double sA, double sB, double sC,
                     double A, double B, double C);

    bool validateTriangle();
    double getArea();
    double getPerimeter();
    std::string triangleType();
};
```

Image 1. Code Snippet for Function Declaration

Analysis

The image above shows the code snippet of the function declaration of the program which shows all the data types and all the functions that I created for each task.

```cpp
//MAIN
int main() {

    double kelvin;

    Triangle tri(5, 6, 7, 40, 50, 90);

    std::cout << "Task #1 Swapping two numbers." << std::endl;
    numSwap();

    std::cout << "\nTask #2 Kelvin to Fahrenheit Conversion" << std::endl;
    std::cout << "Enter Temperature you want to convert in Kelvin: ";
    std::cin >> kelvin;
    std::cout << "\nTemperature converted to Fahrenheit: "
              << kelvinToFahrenheit(kelvin) << " Fahrenheit" << std::endl;

    std::cout << "\nTask #3 Distance Between Two Points" << std::endl;
    calculateDistance();
    std::cout << "\nTask #4 Modifying Triangle Class and getting Area, Perimeter, Triangle Type." << std::endl;
    if (tri.validateTriangle()) {
        std::cout << "The triangle is valid.\n";
        std::cout << "Perimeter : " << tri.getPerimeter() << std::endl;
        std::cout << "Area      : " << tri.getArea() << std::endl;
        std::cout << "Type      : " << tri.triangleType() << std::endl;
    }
    else {
        std::cout << "The triangle is NOT valid.\n";
    }

    return 0;
}
```

Image 2. Code Snippet for Main Function

Analysis

Basically, this image contains the main function of the program. This is where user input is required and the different functions are called to be used by the user.

```cpp
//Task 1
void numSwap() {


    int a, b, temp;

    std::cout << "Input Values for A and B\n";
    std::cout << "A = ";
    std::cin >> a;
    std::cout << "B = ";
    std::cin >> b;

    std::cout << "\nCurrent Values of A and B\n";
    std::cout << "A = " << a << std::endl;
    std::cout << "B = " << b << std::endl;
//Setting temporary variable for switching
    temp = a;
    a = b;
    b = temp;

    std::cout << "\nValues after swapped" << std::endl;
    std::cout << "A = " << a << std::endl;
    std::cout << "B = " << b << std::endl;
}
```

Image 3. Code Snippet for Task 1 function

Analysis

This image shows the function for task 1 which is the number swapping function. The temp stands as the temporary variable which makes it possible for me to switch the values by designating the value of another value to it. It acts like a placeholder.

```
Task #1 Swapping two numbers.
Input Values for A and B
A = 2
B = 3

Current Values of A and B
A = 2
B = 3

Values after swapped
A = 3
B = 2
```

Image 4. Output for Task 1

```
//Task 2
double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9 / 5 + 32;

}
```

Image 5. Code Snippet for Task 2

Analysis

This image shows the function for the conversion rate of kelvin to fahrenheit. This basically is just a simple mathematical formula which I used in the task.

```
Task #2 Kelvin to Fahrenheit Conversion
Enter Temperature you want to convert in Kelvin: 25

Temperature converted to Fahrenheit: -414.67 Fahrenheit
```

Image 6. Output for Task 2

```cpp
// Task 3
void calculateDistance() {

    double x1, x2, y1, y2;

    std::cout << "Enter x1: ";
    std::cin >> x1;

    std::cout << "Enter y1: ";
    std::cin >> y1;

    std::cout << "Enter x2: ";
    std::cin >> x2;

    std::cout << "Enter y2: ";
    std::cin >> y2;

    std::cout << "\nDistance Calculated:" << std::endl;
    std::cout << "X1 : " << x1 << std::endl;
    std::cout << "Y1 : " << y1 << std::endl;
    std::cout << "X2 : " << x2 << std::endl;
    std::cout << "Y2 : " << y2 << std::endl;

    std::cout << "Distance is = " << distanceCalculator(x1, x2, y1, y2) << std::endl;
}

double distanceCalculator(double x1, double x2, double y1, double y2) {

    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));

}
```

Image 7. Code Snippet for Task 3

Analysis

The image above shows the code snippet for the distance formula task. As you can see, the system requires user input and once the user input is set, the distance formula calculation is initiated by the system.

```
Task #3 Distance Between Two Points
Enter x1: 3
Enter y1: 2
Enter x2: 3
Enter y2: 2

Distance Calculated:
X1 : 3
Y1 : 2
X2 : 3
Y2 : 2
Distance is = 0
```

Image 8. Output for Task 3

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```
// Task 4
//Constructor
Triangle::Triangle(double sA, double sB, double sC,
                   double A, double B, double C)
{
    sideA = sA;
    sideB = sB;
    sideC = sC;

    angleA = A;
    angleB = B;
    angleC = C;

    totalAngle = A + B + C;
}
```

Image 9. Code Snippet for Constructor

Analysis

The code snippet is the initializer wherein a permanent variable is assigned a designated temporary variable. After that, it adds all the angles in order to get the total angle.

```
//Setter
void Triangle::setTriangle(double sA, double sB, double sC,
                           double A, double B, double C)
{
    sideA = sA;
    sideB = sB;
    sideC = sC;

    angleA = A;
    angleB = B;
    angleC = C;

    totalAngle = A + B + C;
}
```

Image 10. Code Snippet for Setter

Analysis

This image shows how we can reuse the triangle function that we already set later on in the program.

```
//Validator
bool Triangle::validateTriangle()
{
    return totalAngle == 180;
}
```

Image 11. Code Snippet for Triangle Validator

```
//Perimeter Calculation
double Triangle::getPerimeter()
{
    return sideA + sideB + sideC;
}
```

Image 12. Code Snippet for Perimeter Calculator

```
//Area Calculation
double Triangle::getArea()
{
    double s = getPerimeter() / 2;

    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}
```

Image 13. Code Snippet for Area Calculator

```
//Triangle Type
std::string Triangle::triangleType()
{
    if (angleA < 90 && angleB < 90 && angleC < 90)
        return "Acute-Angled";

    else if (angleA > 90 || angleB > 90 || angleC > 90)
        return "Obtuse-Angled";

    else
        return "Others";
}
```

Image 14. Code Snippet for Triangle Type

```
Task #4 Modifying Triangle Class and getting Area, Perimeter, Triangle Type.
The triangle is valid.
Perimeter : 18
Area      : 14.6969
Type      : Others
```

Image 15. Output for Task 4

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.